

Invalid Input Data and Conformity Testing for an Object-Oriented Specification

Khadija Louzaoui

Laboratory of Informatics, Systems and Optimization (ISO)
Department of Computer Science, Faculty of Sciences
Ibn-Tofail University, Kenitra, Morocco

Khalid Benlhachmi

Laboratory of Informatics, Systems and Optimization (ISO)
Department of Computer Science, Faculty of Sciences
Ibn-Tofail University, Kenitra, Morocco

Abstract—The objective of this paper is to introduce a new approach that can be used to add detail to a system requirements specification. This technique is based on invalid input values and can help discover problems in system requirements.

The principal idea of our work is based on a constraints model and an equivalence partitioning of input domains for programs in the Object Oriented (OO) paradigm for detecting different classes of errors. The main contribution of our approach is the use of invalid data which do not satisfy the precondition constraints for testing the robustness of entities in an OO model. Indeed, the first objective of the proposed work is to develop a theoretical model of constraint in order to test the conformity of classes. The second objective of our approach is to detect anomalies in invalid input data which induce valid output constraints. The implementation of this approach is based on a random generation of test data and analysis by formal proof.

Keywords—Formal specification; conformity testing; robustness testing; valid data; invalid data; test data generation; inheritance; constraint resolution.

I. INTRODUCTION

The principal value of using formal specification methods in the software process is that it forces an analysis of the system requirements at an early stage. Correcting errors at this stage is cheaper than modifying a delivered system. Formal specification techniques are most cost-effective in the development of critical systems where safety, reliability and security are particularly important. They may also be used to define the contracts between users and programs. Indeed, in an Object Oriented (OO) model a formal specification is the set of input and output constraints used to describe by logical equations the properties of programs (Fig.1). Precondition (P), postcondition (Q) and invariant (Inv) are the main used constraints in an OO specification (Fig.2), and can be written by using specific languages as OCL [1] and JML [2].

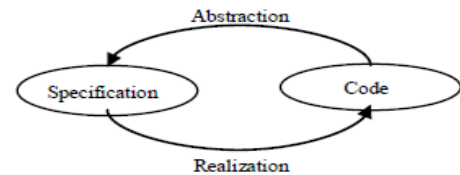


Fig.1. Relationship between code and specification

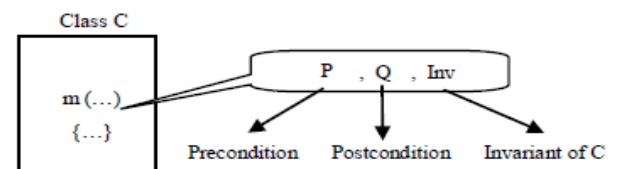


Fig.2. Constraints of the method $m()$

Design by constraints is introduced by Meyer [3,4] and defines the contract between users and programs. In this context the conformity of a program in an OO paradigm means that the output constraints are satisfied if the input constraints are satisfied for all invocation of the program under test (Fig. 3). In an OO testing, the test data generation is based on the valid input data that satisfy the precondition constraints, and is used to testing the conformity of a class implementation from its specification. The generated data from the values domain with respect to the well defined constraints can be generated randomly or using constraints resolution.

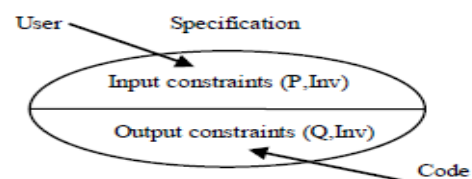


Fig.3. User and code contracts

The main contribution of this work is the definition of a complementary test based on invalid input data that do not satisfy the precondition constraint of the method under test. The idea is to integrate the invalid data in the testing process for strengthening the conformity testing of OO classes. Indeed, the analysis of program behavior towards invalid input data is used to enrich the concept of test and to detect design anomalies.

The test oracles do not define the behavior of implementations towards invalid input data and they are restricted to conformity testing by generating only data meeting the precondition constraints. In our approach we complete the testing process by introducing a test based on the invalid data to measure the robustness of OO programs. In this context, we specify the anomalies of constraints which are due to input data that meet the contract of program under test even if input constraints are invalid (Fig.4). These input data induce two problems:

- A problem on the relevance of specifications: incomplete precondition (satisfied contract in outside of the valid domain (Fig.4)).
- A problem on the program robustness (the code can ensure output contracts for an invalid input value (Fig.4)).

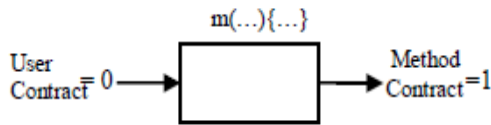


Fig.4. Case of anomalies in contracts

This paper is organized as follows: in section 2 we present related works and similar approaches for generating test data from a formal specification, in sections 3 and 4 we describe theoretical aspects of our test process, and we define our test formal model of constraints, partitions deduced from the formal model. In section 5 we present how the formal model and domain partitioning can be used to generate data for testing the conformity of methods in an OO class, in section 6 we present our approach of robustness testing that strengthens the conformity testing, and we show how the robustness approach uses the domains partition for generating test data, and finally we describe our approach with an example of conformity and robustness testing for an OO model.

II. RELATED WORK

Most works have studied the problem of test data generation and formal specification for OO programs. These works show how the programs conformity can be tested by using white box testing that takes into account the internal mechanism of a system or black box testing that ignores the internal mechanism of a system or component and focuses solely on the outputs generated in response to selected inputs and execution conditions.

In [1], the authors present a method based on the constraints resolution for test cases generation with error anticipation in the methods specification. In [2], the approach presents a model-based framework for the symbolic animation of object-oriented specifications. This technique can be applied to Java Modeling Language (JML) specifications, making it possible to animate Java programs that only contain method interfaces and no code. In [5], the authors propose a generation of test data from formal specifications using the Object Constraint Language (OCL).

In [6] the authors present a test derivation and selection method based on a model of communicating processes with inputs, outputs and data types, which is closer to actual implementations of communication protocols.

In [7], we have developed a basic model for the concept of methods similarity, the test is based only on a random generation of input data. In [8], we have generalized the basic model of similar behaviors using constraints propagation, equivalence partitioning and formal proof.

In [9], we have based on the similar behaviors for testing the conformity of overriding methods in inheritance. This approach shows that the test cases developed for testing an original method can be used for testing its overriding method in a subclass and then the number of test cases can be reduced considerably.

In [10], the authors propose a randomly generation of test data from a JML specification of class objects. They classify the methods and constructors according to their signature (basic and extended constructors, mutator, and observer) and for each type of individual method of class, a generation of test data is proposed. In [11], the authors use the constraints resolution principle to reduce the values of testing data for limited domain types and use a random generation for other data types. In [12] the authors propose a theoretical framework for model based robustness testing together with an implementation within the If validation environment.

In [13], the paper describes specially the features for specifying methods, related to inheritance specification; it shows how the specification of inheritance in JML forces behavioral sub-typing. In order to study the effectiveness and performance of random test techniques, the works presented in [14] propose to test the conformity of methods using experimental proofs. In [15], the authors developed a testing tool called JCrasher for random testing of Java classes. Tool oriented, the authors propose in [16] a configurable unit testing tool of Java classes specified in JML.

In [17] the paper gives a description of testing methods based on algebraic specifications, and a brief presentation of some tools and case studies, and presents some applications to other formal methods involving data types. In [18] the authors have studied the multi-objective test data generation problem. The authors in [19] present a robustness modeling methodology that allows modeling robustness behavior as aspects. The goal is to have a complete and practical methodology that covers all features of state machines and aspect concepts necessary for model-based robustness testing. In [20] the authors present a survey of some of the most prominent techniques of automated test data generation, including symbolic execution, model-based, combinatorial, adaptive random and search-based testing.

Test oracles and current standards of verification do not define implementations behaviors towards invalid input data and they are restricted to conformity testing by generating only data meeting the precondition constraints. In our approach we complete the testing process by introducing a test based on the invalid data to measure the robustness of OO programs.

III. FORMAL MODEL OF CONSTRAINT

This section presents the definition of a new constraint for modeling the specification of methods in OO classes. In this context, we propose a constraint model that contains the precondition P , postcondition Q , and the invariant Inv (Fig.2) into a single logical formula. This model should translate algebraically the contract between users and programs.

Let C be a class, and m be a method of n arguments $x = (x_1, x_2, \dots, x_n)$. We define for each argument x_i its domain of values E_i . We put $E = (E_1 \times E_2 \times \dots \times E_n)$ where E is the domain of input vectors of the method m .

The main idea of this relationship is based on the following interpretation:

If the user respects his own part of the contract by invoking a method with arguments satisfying the pre-condition P , then the method must necessarily satisfy the post-condition Q after the call. Concerning the invariant, it must be satisfied before and after the method call.

Definition1

We define the constraint H of a method m of class C as a property of the pair (x, o) (x is the input vector and o is the receiver object) such that:

$$H(x, o) : [P(x, o) \wedge Inv(o_{(bef)})] \Rightarrow [Q(x, o) \wedge Inv(o_{(aft)})], (x, o) \in E \times I_c$$

Where:

- I_c is the set of instances of the class C .
- $o_{(bef)}$ is the class object o in the state before the calling of the method $m()$.
- $o_{(aft)}$ is the class object o in the state after the calling of the method $m()$.

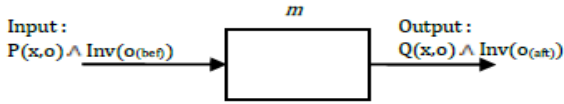


Fig.5. Input-Output constraints of a method $m()$

The logical implication in the proposed formula means: each call of method with (x, o) satisfying the pre-condition P and the invariant Inv before the call, (x, o) must necessarily satisfy the post-condition Q and the invariant after the call (Fig. 5). In the context of OO modeling, this constraint can be reduced if we consider that all invocations of method m are done with a valid object o satisfying the invariant (instantiated by a valid constructor). We deduce that the invariant of object o in the State-Before is satisfied: $Inv(o_{(bef)}) = I$.

Therefore:

$$H(x, o) : P(x, o) \Rightarrow [Q(x, o) \wedge Inv(o_{(aft)})], (x, o) \in E \times I_c \quad (1)$$

IV. PARTITION ANALYSIS

The analysis of the input domain of a method is a crucial step in the implementation of tests. Indeed, this analysis allows dividing the domains to locate the potential data which can affect the test problem in order to identify the anomalies origin. In this section, the constraint H defined above is used in the generation of domain partitions for each type of methods

according to the classification proposed in [10]. Indeed, this classification includes four types of methods (C_b : basic constructor, C_e : extended constructor, M : mutator and O : observer)(Fig.7).

In this context, we divide firstly the input domain $E \times I_c$ of the method under test into two sets A and B :

$$A = \{(x, o) \in E \times I_c / H(x, o) = 1\} \text{ and}$$

$$B = \{(x, o) \in E \times I_c / H(x, o) = 0\}$$

Then, A can be divided into two subsets A_1 and A_2 :

$$A_1 = \{(x, o) \in E \times I_c / (P(x, o), Q(x, o), Inv(o)) = (1, 1, 1)\}$$

$$A_2 = \{(x, o) \in E \times I_c / (P(x, o), Q(x, o), Inv(o)) = (0, ?, ?)\}$$

We divide A_2 (for a false pre-condition) to 4 domains $A_{2,1}$, $A_{2,2}$, $A_{2,3}$ et $A_{2,4}$.

B can be divided into 3 subsets B_1 , B_2 and B_3 (Fig.6).

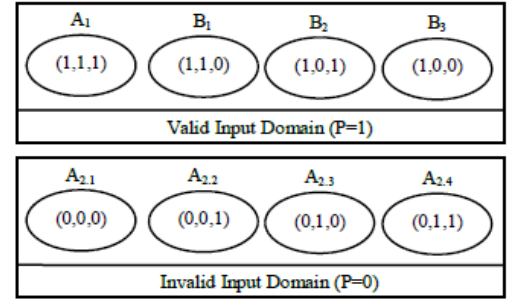


Fig.6. Input Domain partitioning

(A_1, B_1, B_2, B_3) is the partition whose elements satisfy the precondition constraint (i.e. the partition of the valid domain) and $(A_{2,1}, A_{2,2}, A_{2,3}, A_{2,4})$ is the partition whose elements do not satisfy the precondition constraint (i.e. the partition of the invalid domain)(Fig.6)

```

1. class Account
2. {
3.     private int bal; // bal is the account balance
4.     // cb_Account() is a basic Constructor Cb
5.     public static Account cb_Account (int x){
6.         Account o = new Account(); o.bal=x; return o ;
7.     }
8.     // ce_Account() is an extended Constructor Ce
9.     public static Account ce_Account (Account x){
10.        Account o = new Account(); o.bal=x.bal; return o ;
11.    }
12.    public void setBal (int x){this.bal=x;}
13.    // getBal() is an observer O
14.    public int getBal () {return this.bal;}
15.    /* transfer(x1,x2) is a mutator */
16.    public void transfer (int x1, Account x2)
17.    {this.bal=this.bal - x1; x2.bal=x2.bal + x1;}
18.    public void withdraw (int x1){...}
19.    public void deposit (int x1){...}
20. }

```

Fig.7. Java Implementation of the Account class

V. CONFORMITY TESTING

In this section we use the model of constraint H and partitioning of input domains to describe the concept of conformity of a method in a class of objects. Indeed, the purpose of this section is to model the conformity of a class by a logical equation and implement algorithms for generating test data.

A. Formal model of conformity testing

The conformity of a method $m()$ in an OO paradigm means that the output constraints are satisfied **if** the input constraints are satisfied for all invocation of the method under test.

We consider a method m of class C where o is the receiver object and x the parameters vector $((x, o) \in E \times I_c)$

Definition 2

A method m is valid or conforms to its specification if for each (x, o) , the constraint H is satisfied:

$$\forall (x, o) \in E \times I_c : H(x, o) \quad (2)$$

In other words, for all elements of the input domain: If the precondition P is satisfied then the postcondition Q and the invariant Inv must be satisfied.

In order to check that the method m does not conform to its specification, it is sufficient to find an input value that satisfies the precondition but does not satisfy output constraints:

$$\exists (x, o) \in E \times I_c : P(x, o) \wedge (\overline{Q(x, o)} \vee \overline{Inv(o)})$$

B. Algorithms of conformity testing

The main goal of the testing algorithm is the use of the constraint model $H: P \Rightarrow (Q \wedge Inv)$ and the valid input domain partitioning (A_1, B_1, B_2, B_3) for checking if the method under test meets its specification. The algorithm execution of conformity test stops when the constraint H becomes False ($H(x, o) = 0$) or when we reach the threshold of test with H True (Fig.8). N represents the number of times that the test is executed.

```

1. do{
2.   do{
3.     for ( x1 in parameter(M))
4.       {x1 = generate ( E1) ; }
5.       x= (x1, x2, ..., xn) ;
6.       o = generate_object();
7.       }while (!P(x, o));
8.       invoke "o.M(x)"
9.       if (Q(x, o) && Inv(o))
10.        A1.add(x, o); /* The set A1 does not
                        contain the same elements*/
11.      elseif ( Q(x, o) && !Inv(o))
12.        B1.add(x, o);
13.      elseif (!Q(x, o) && Inv(o))
14.        B2.add(x, o);
15.      else
16.        B3.add(x, o)
17.    }while (A1.size() < N && B1.isEmpty() &&
           B2.isEmpty() && B3.isEmpty());

```

Fig.8 . Conformity test algorithm of a mutator M

C. Evaluation

We consider an example for testing the conformity of the method *transfer* of the class *Account* (Fig. 7). As is indicated above, the conformity test of mutator requires passing through

a test of a basic constructor in order to use valid objects in the testing process.

The constraint H of the method *transfer* (Fig.7) in an algebraic specification is shown in the Figure 9 ($x = (x_1, x_2)$, $o_{(a)}$ and $o_{(b)}$ are respectively the object o after and before the call of the transfer method):

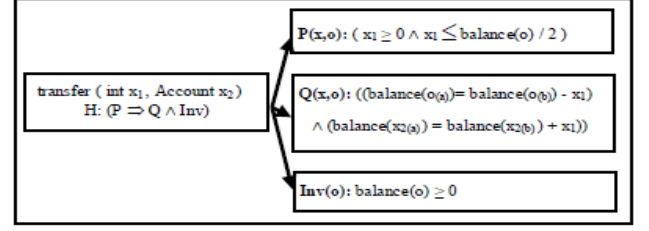


Fig.9. Constraints of the method *transfer*

We generate randomly x_i values in the interval $]-200, 200[$ and $N=100$ (Table 1).

TABLE I. RESULT OF A CONFORMITY TEST OF THE TRANSFER METHOD

Iteration number	(x1,x2)	O	P(x,o)	(x,o) ∈	H(x,o)
1	(21, Account(34))	Account(74)	1	A ₁	1
2	(59, Account(98))	Account(164)	1	A ₁	1
3	(48, Account(109))	Account(182)	1	A ₁	1
...	...	...	...	...	...
...	...	...	...	...	...
...	...	...	...	...	...
98	(36, Account(7))	Account(107)	1	A ₁	1
99	(73, Account(30))	Account(199)	1	A ₁	1
100	(19, Account(77))	Account(185)	1	A ₁	1

The test result shows that for 100 iterations the constraint H is always True ($H = 1$), this leads to the conclusion that the *transfer* method meets its specification (Table 1).

VI. ROBUSTNESS TESTING

The robustness testing is an important step in the verification process for an OO specification and can detect anomalies in invalid input data that induce valid output constraints. In this section we present a constraint model and algorithms for testing the robustness of OO programs.

A. Formal model of robustness testing

In our approach, the robustness verification of a method $m()$ in an OO paradigm means that the output constraints are satisfied **if and only if** the input constraints are satisfied for all invocation of the method under test. In this way, the invalid input data must induce only invalid output constraints (Fig.10).

Definition 3 (Robust method)

A method m is robust according to its specification if it satisfies the following conditions:

- It conforms to its specification.
- For each invalid input data (x, o) (does not satisfy the precondition $P(x, o) = 0$), the postcondition Q and the

invariant Inv should not be both valid in output
 $(Q(x, o) = 0 \vee Inv(o) = 0)$.

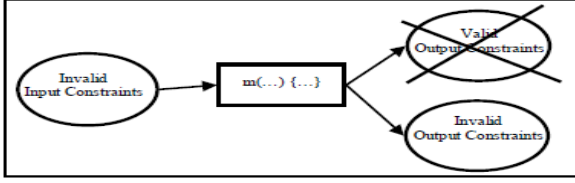


Fig.10. Principle of robustness testing

On the theoretical level, we are looking for strengthening the current constraint H in order to integrate this type of test. Indeed, the constraint H defined above for conformity test takes in the robustness approach the following form:

$$H_{robustness}(x, o) : [P(x, o) \Leftrightarrow (Q(x, o) \wedge Inv(o))], (x, o) \in E \times I_c$$

In this test, we assume that with an invalid input, we can expect only invalid constraints: any valid result coming from an invalid input indicates the presence of a robustness problem into the method implementation.

The testing approach consists of two steps of testing:

- Main testing or conformity testing.
- Complementary testing or robustness testing concerns only the methods where the conformity testing is validated.

Theorem

The method m is robust with respect to its specification if:

$$\forall (x, o) \in E \times I_c : H_{robustness}(x, o) = 1$$

B. Algorithms of Robustness Testing

The main goal of robustness algorithms is the use of the constraint model $H_{robustness} : P \Leftrightarrow (Q \wedge Inv)$ and the invalid input domain partitioning $(A_{2.1}, A_{2.2}, A_{2.3}, A_{2.4})$ for checking if the method under test is robust relatively to its specification. The algorithm execution of robustness test stops when the constraint $H_{robustness}$ becomes False ($H_{robustness}(x, o) = 0$) or when we reach the threshold of test with $H_{robustness}$ True (Fig.11). We assume that the method under test is in conformity with its specification. N represents the number of times that the test is executed (N is an input constant of the algorithm). In this algorithm we generate randomly input values that do not satisfy the precondition constraint.

```

1. do{
2. do{
3. for ( xi in parameter(M) )
4. {xi = generate ( Ei ) ; }
5. x= (x1, x2, ..., xn) ;
6. o = generate_object ( ) ;
7. }while (P(x, o));
8. invoke "o.M(x)"
9. if ( !Q(x, o) && !Inv(o) )
10.  A21.add(x, o);
11. elseif ( !Q(x, o) && Inv(o) )
12.  A22.add(x, o);
13. elseif ( Q(x, o) && !Inv(o) )
14.  A23.add(x, o);
15. else
16.  A24.add(x, o)
17. }while (A21.size() < N && A22.size() < N &&
      A23.size() < N && A24.isEmpty());

```

Fig.11. Robustness test algorithm of a mutator M

C. Evaluation

We consider the *transfer* method of the *Account* class (Fig.7). The conformity test presented in the last section shows that this method is conforming to its specification. An example of a robustness test of the *transfer* method where $N=100$ and x_1 into the interval $]-200, 200[$ gives the following results :

TABLE II. RESULT OF A ROBUSTNESS TEST OF THE TRANSFER METHOD

Iteration number :	(x_1, x_2)	O	$P(x, o)$	$H_{robustness}(x, o)$
1	(136, Account(35))	Account(32)	0	1
2	(100, Account(67))	Account(78)	0	1
3	(46, Account(120))	Account(41)	0	1
4	(87, Account(74))	Account(140)	0	0

In the iteration 4, we have: $H_{robustness}(x, o) = 0$. Indeed, our implementation cannot reject this situation and consequently the method *transfer* under test which is conforming to its specification, is considered not robust relatively to the same specification.

VII. CONCLUSIONS AND FUTURE WORK

This paper introduces an approach to generate test data from formal specifications in OO software testing. The first result of our work is a constraint model for testing the conformity of OO methods. The second result of our approach is the use of an equivalence partitioning technique that can be used to reduce the number of test cases by uncovering various classes of errors. The third and main result of this work is a negative testing that completes the testing process by introducing the invalid data to measure the robustness of OO programs.

The limitation of this approach is that it is applied only to OO classes without considering the connections between these classes. This the reason why our future works are now oriented to generalize our formal model for testing the robustness of overriding methods in sub-classes by using constraints propagation in inheritance operation.

REFERENCES

- [1] B. K. Aichernig and P. A. P. Salas. "Test case generation by OCL mutation and constraint solving". In Proceedings of the International Conference on Quality Software, Melbourne, Australia, September 19-20, 2005, pages 64–71, 2005.
- [2] F. Bouquet, F. Dadeau, B. Legeard, and M. Utting. "Symbolic animation of JML specifications". In International Conference on Formal Methods, volume 3582 of LNCS, pages 75–90. Springer-Verlag, July 2005.
- [3] MEYER, Bertrand. "Applying design by contract". Computer, 1992, vol. 25, no 10, p. 40-51.
- [4] Bertrand Meyer "Design by Contract Object and Component Technology Series", Prentice Hall PTR, 2000.
- [5] Mohammed Benattou, Jean-Michel Bruel, and Nabil Hameurlain. "Generating Test Data from OCL Specification". In Proceedings of the ECOOP'2002 Work-shop on Integration and Transformation of UML models (WITUML'2002), 2002.

- [6] Lestiennes, Grégory et Gaudel, Marie-Claude. "Testing processes from formal specifications with inputs, outputs and data types". In : Software Reliability Engineering, ISSRE 2003. Proceedings. 13th International Symposium on. IEEE, 2002. p. 3-14.
- [7] Khalid Benlhachmi, and M.Benattou. "A Formal Model of Similarity Testing for Inheritance in Object-Oriented Software". In Proceedings of the IEEE International Conference (CIST'2012) ,October 24-26,2012, Fez, Morocco. Pages 38-42.
- [8] Khalid Benlhachmi, and M.Benattou, "Similar Behaviors and conformity testing in Inheritance for an object oriented model", In IADIS International Journal on Computer Science and Information Systems ,Vol. 9, No. 1, 2014. Pages 30-42.
- [9] Khalid Benlhachmi, and M.Benattou, "A Formal Model of Conformity Testing of Inheritance for Object Oriented Constraint Programming", In International Journal of Software Engineering and Its Applications, Vol 7, No 5, 2013. Pages 209-220.
- [10] Yoonsik Cheon and Carlos E. Rubio-Medrano. "Random Test Data Generation for Java Classes Annotated with JML Specifications". In Proceedings of the 2007 International Conference on Software Engineering Research and Practice, Volume II, June 25-28, 2007, Las Vegas, Nevada, pages 385-392
- [11] Yoonsik Cheon, Antonio Cortes, Martine Ceberio, and Gary T. Leavens. " Integrating Random Testing with Constraints for Improved Efficiency and Diversity". In Proceedings of SEKE 2008, The 20-th International Conference on Software Engineering and Knowledge Engineering, July 1-3, 2008, San Francisco, CA, pages 861-866, July 2008.
- [12] Fernandez, Jean-Claude, Mounier, Laurent, et Pachon, Cyril. "A model-based approach for robustness testing". In : Testing of Communicating Systems. Springer Berlin Heidelberg, 2005. p. 333-348.
- [13] G.T.Leavens,"JML's Rich Inherited Specification for Behavioral Subtypes" Department of Computer Science Iowa State University (2006) August 11.
- [14] I. Ciupa, A. Leitner, M. Oriol, and B. Meyer. "Experimental assessment of random testing for object-oriented software". In International Symposium on Software Testing and Analysis, pages 84–94. ACM, 2007.
- [15] C. Csallner and Y. Smaragdakis. "JCrasher : an automatic robustness tester for Java". Software Practice and Experience, 34(11):1025–1050, Sept. 2004
- [16] C. Oriat. Jartege: "A tool for random generation of unit tests for Java classes". In International Conference on the Quality of Software Architectures, volume 3712 of LNCS, pages 242–256. Springer-Verlag, Sept. 2005.
- [17] Gaudel, Marie-Claude et Le Gall, Pascale."Testing data types implementations from algebraic specifications". In : Formal methods and testing. Springer Berlin Heidelberg, 2008. p. 209-239.
- [18] Ferrer, Javier, Chicano, Francisco, et Alba, Enrique. "Evolutionary algorithms for the multi-objective test data generation problem". Software: Practice and Experience, 2012, vol. 42, no 11, p. 1331-1362.
- [19] Ali, Shaukat, Briand, Lionel C. et Hemmati, Hadi. "Modeling robustness behavior using aspect-oriented modeling to support robustness testing of industrial systems". Software & Systems Modeling, 2012, vol. 11, no 4, p. 633-670.
- [20] Anand, Saswat, Burke, Edmund K., Chen, Tsong Yueh, et al. "An orchestrated survey of methodologies for automated software test case generation". Journal of Systems and Software, 2013, vol. 86, no 8, p. 1978-2001.